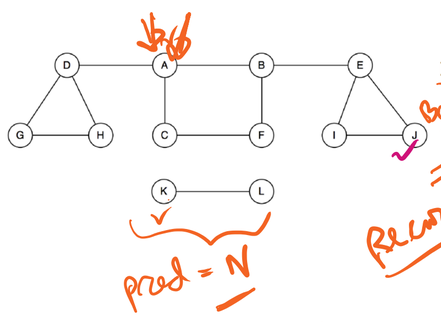
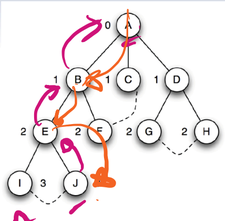
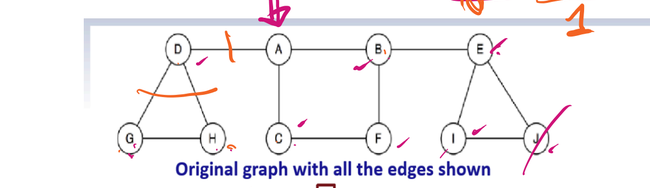
Breadth-first search - Printing a path from *s* to *v*

- BFS tree shows how each vertex was reached from the **source**.
- The **predecessor info** (u.π) stores the **previous node** on the path.
- Using this info, we can easily **reconstruct or print the path** from the source s to any node v.

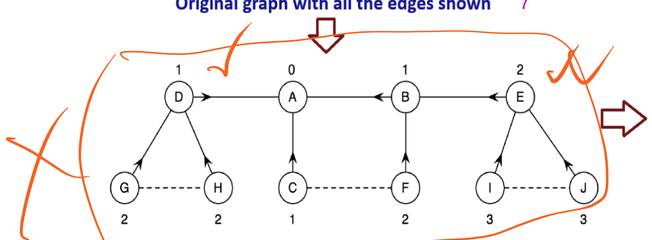



```
Print-Path(G, s, v)
  if v == s
    print s
  elseif v.p == NIL
    print "no path from s to v"
  else
    Print-Path(G, s, v.p)
    print v
```

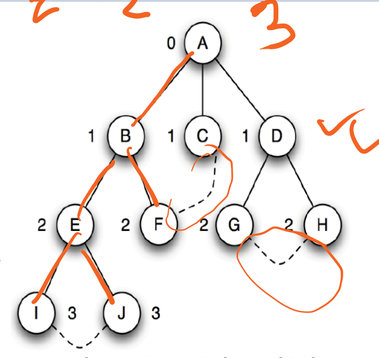




Original graph with all the edges shown



BFS tree with node distances from the source
Only path of traversal shown



Redrawn BFS tree in hierarchical tree format with distances from the source.

Consider the given **directed weighted graph**, where each vertex is labeled with a numerical value, and each directed edge has an associated weight. You are required to perform a **special Breadth-First Search (BFS)** traversal starting from **vertex 5**. In this special BFS, when a vertex has more than one neighbor, the next vertex is chosen based on the smallest sum of the vertex value and the edge weight. In other words, the neighbor with the minimum (vertex value + edge weight) is visited first.

You must (a) **show the step-by-step simulation** of the traversal. After completing the traversal, write the (b) **final BFS traversing order** and (c) **list down any vertices that remain unvisited.**

You are **not required to write any code** for this problem. All the necessary steps of the simulation must be shown.

